

ECE 1387 Lecture #3

Sept 26, 2025

Routing continued...

- How's AI going?

Maze routing : route of length p

			3			
		3	2	3		
	3	2	1	2	3	
3	2	1	0	1	2	3
	3	2	1	2	3	
		3	2	3		
			3			

How many cells
have label

0: 1

1: 4

2: 8

3: 12

⋮

$p: 4p$

So, for path
length of
 p , # cells
visited:

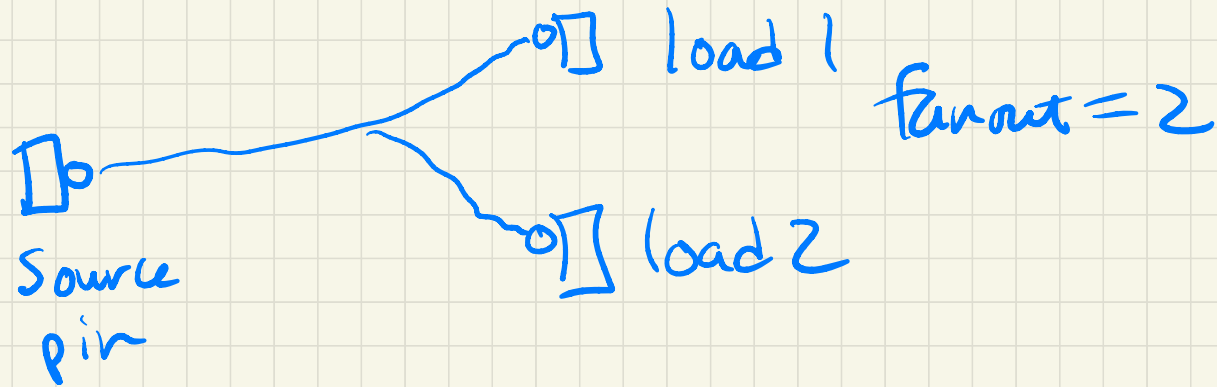
$$4 \sum_{i=0}^p i$$

$$4 \cdot \frac{(p)(p+1)}{2}$$

So, for path length of P
runtime of maze routing is
 $O(P^2)$

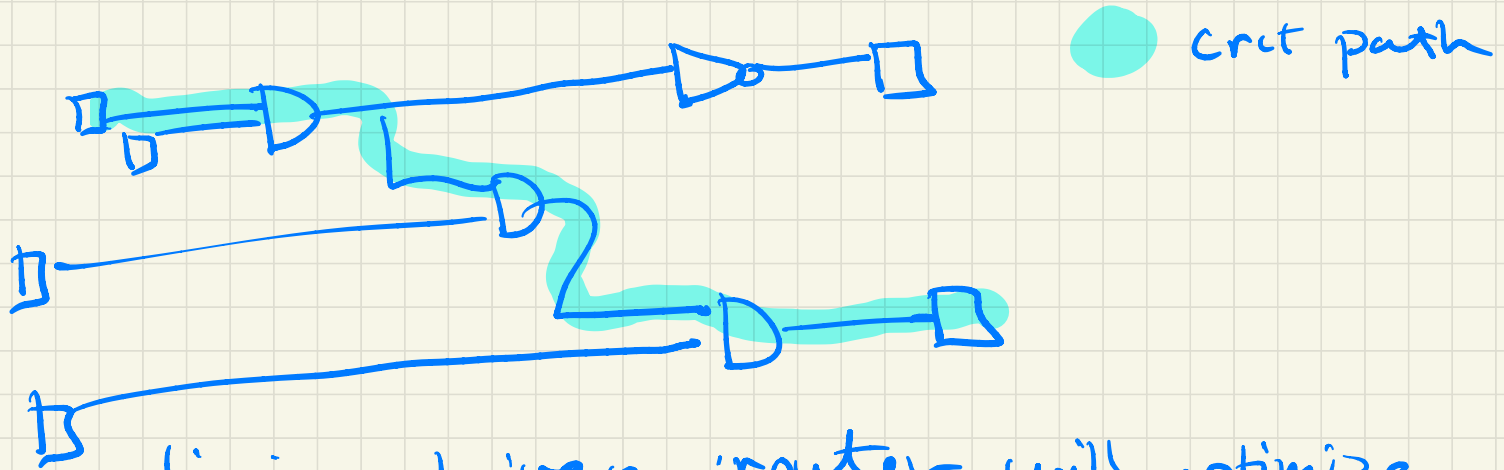
More complex routing

1) Has to deal with fanout



one way: first route one load
then, for 2nd load, we can initialize
expansion list to be existing route
for first load

2) Good routers optimize for speed

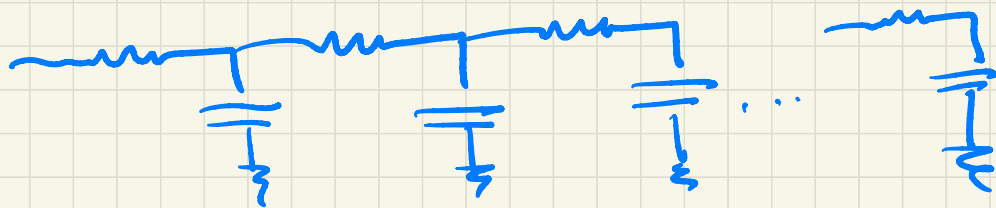


timing-driven router will optimize
delay of connections on critical
path

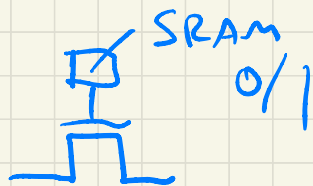
Interconnect Delay Modeling

FPGA distrib RC

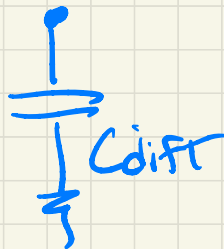
wire



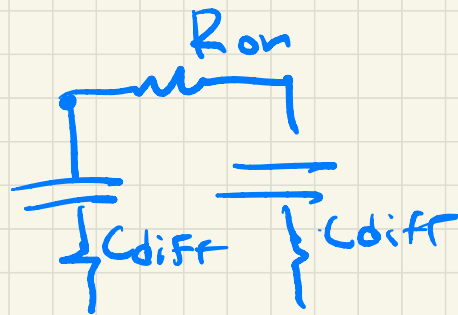
Switches

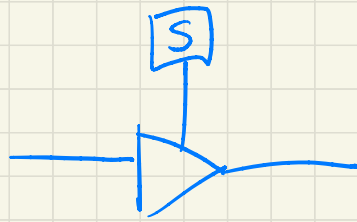


off



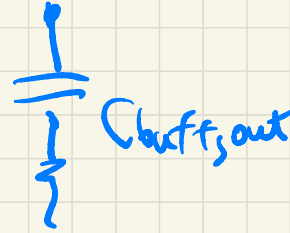
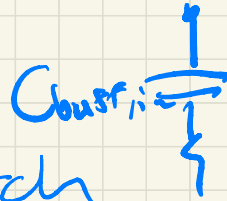
on



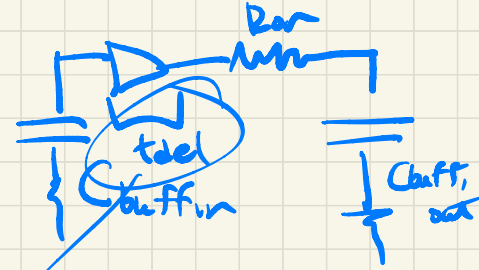


Buffered switch

off



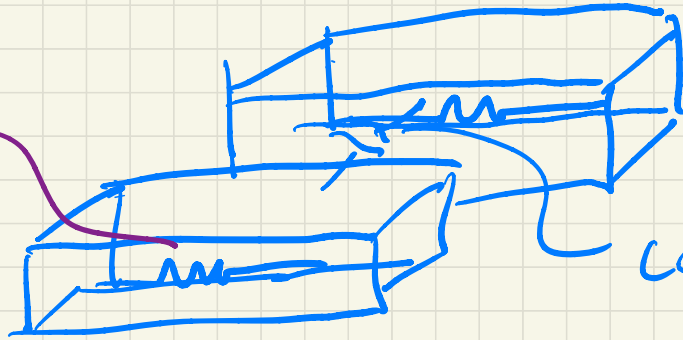
on



intrinsic
constant buffer
delay

What about in ASICs?

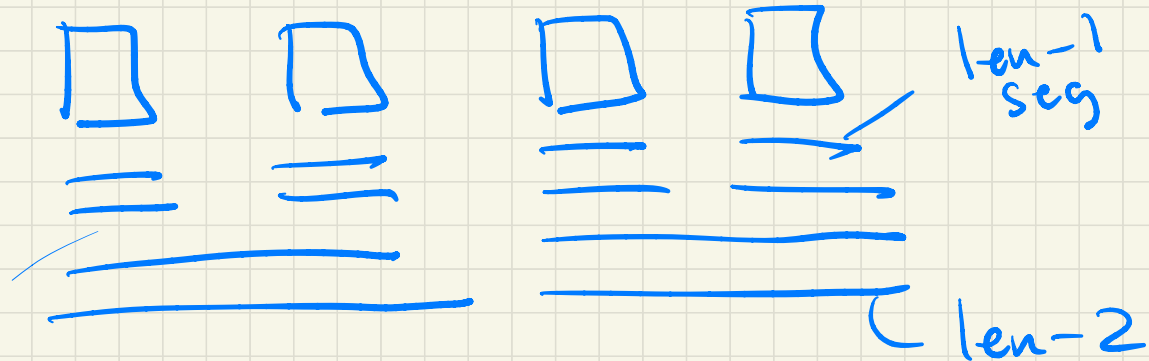
metal wire



coupling cap

metal wires have resistance & capacitance (to neighbouring wires in same layer or adjacent layers of metal)

More complex routing architecture
Modern FPGAs have wire segments
of different length



Router decides which seg.
types to use when routing a
connection

Now, routing is a more complicated problem...

PathFinder does a good job
of handling these complexities

- U. Washington mid-90s
- basis of Xilinx / Altera commercial routers -



PathFinder: A Negotiation-Based Performance-Driven Router for FPGAs

Larry McMurchie and Carl Ebeling
Dept. of Computer Science and Engineering
University of Washington, Seattle, WA

Abstract

Routing FPGAs is a challenging problem because of the relative scarcity of routing resources, both wires and connection points. This can lead either to slow implementations caused by long wiring paths that avoid congestion or a failure to route all signals. This paper presents PathFinder, a router that balances the goals of performance and routability. PathFinder uses an iterative algorithm that converges to a solution in which all signals are routed while achieving close to the optimal performance allowed by the placement. Routability is achieved by forcing signals to negotiate for a resource and thereby determine which signal needs the resource most. Delay is minimized by allowing the more critical signals a greater say in this negotiation. Because PathFinder requires only a directed graph to describe the architecture of routing resources, it adapts readily to a wide variety of FPGA architectures such as Triptych, Xilinx 3000 and mesh-connected arrays of FPGAs. The results of routing ISCAS benchmarks on the Triptych FPGA architecture show an average increase of only 4.5% in critical path delay over the optimum delay for a placement. Routes of ISCAS benchmarks on the Xilinx 3000 architecture show a greater completion rate than commercial tools, as well as 11% faster implementations.

1 Introduction

The problem of routing FPGAs can be stated simply as that of assigning signals to routing resources in order to successfully route all signals while achieving a given overall performance. The first goal, complete routing of all signals, is difficult to achieve in FPGAs because of the lack of routing resources. The usual approach to achieving this goal is to minimize the use of routing resources by constructing minimum routing trees for each signal.

Although this reduces the demand for routing resources, signals will still compete for the same resources and the challenge is to find a way to allocate resources so that all signals can be routed. The second goal, minimizing delay, requires the use of minimum delay routes for signals, which in general are much more expensive in terms of routing resources than minimum routing trees. Thus the solution to the entire routing problem requires the simultaneous solution to two interacting and competing subproblems.

The problem of routing FPGAs bears a considerable resemblance to the problem of global routing for custom integrated circuit design and one would hope to be able to apply the same algorithms to FPGAs. However, the two problems differ in several fundamental respects. First, routing resources in FPGAs are discrete and scarce, while they are reasonably continuous in custom integrated circuits. For this reason FPGAs require an integrated approach using both global and detailed routing. A second difference is that the global routing problem for custom ICs is rooted in an undirected graph embedded in Cartesian space. In FPGAs the switches are often directional, and the routing resources connect arbitrary (but fixed) locations, requiring a directed graph which may not be embedded in Cartesian space. Both of these distinctions are important, as they prevent direct application of much of the work that has been done in custom IC routing to FPGAs.

By far the most common approach to global routing of custom ICs is a shortest path algorithm with obstacle avoidance. By itself, this technique usually yields many unroutable nets, which must be rerouted by hand. A plethora of rip-up and retry approaches have been proposed to remedy the deficiencies of this approach ([Dees81], [Linsker84], [Cohn91]). The basic problem with rip-up and retry is that the success of a route is dependent not just on the choice of which nets to reroute, but also on the order in which rerouting is done.

Several papers have described versions of shortest path with rip-up and retry targeted to FPGAs. [Hill91] uses a breadth-first search while performing routes in random order and a

ACM FPGA
'95



CLUMAP: Clustered Mapper for CGRAs with Predication

Omar Ragheb and Jason H. Anderson

Dept. of Electrical and Computer Engineering, University of Toronto
Toronto, ON M5S 3G4, Canada

omar.ragheb@mail.utoronto.ca, janders@eecg.toronto.edu

Abstract

Coarse-grained reconfigurable architectures (CGRAs) have gained popularity as accelerators for compute-intensive kernels. Complex CGRA architectures that support key features such as multi-context and predication are being developed to support a wider range of kernels. However, mapping applications on these complex architectures poses significant challenges. In this paper, we provide an architecture-agnostic clustered mapping technique and a new cost function tailored for simulated-annealing placement. The mapper simplifies placement and routing phases, demonstrating significant speedup for popular CGRA architectures: HyCUBE and ADRES. Additionally, our method demonstrates an increase in mapping success for the ADRES architecture.

Keywords

CGRAs, predication, mapping, CAD, multi-context

ACM Reference Format:

Omar Ragheb and Jason H. Anderson. 2024. CLUMAP: Clustered Mapper for CGRAs with Predication. In *61st ACM/IEEE Design Automation Conference (DAC '24)*, June 23–27, 2024, San Francisco, CA, USA. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3649329.3658269>

1 Introduction

Different accelerator architectures offer different strategies for reducing an application's execution time compared to general-purpose CPUs. The use of graphics processing units (GPUs), where single instruction, multiple data (SIMD) engines give enhanced computing capability, has been extensively investigated for machine-learning applications. Another strategy involves creating application-specific integrated circuits (ASICs) specifically for compute-intensive tasks. However, reconfigurable architectures present another approach to realize application accelerators, as the hardware may be adapted to meet the requirements of the application and reconfigurable architectures are more energy efficient than GPUs. The performance and power efficiency of reconfigurable architectures is attractive compared to fixed-function ASICs.

Field-programmable gate arrays (FPGAs) and coarse-grained reconfigurable architectures (CGRAs) are the two primary reconfigurable architectures. CGRAs fall somewhere between custom ASICs and FPGAs in terms of configurability, performance, power, and area efficiency. CGRAs are programmable hardware fabrics

that consist of a two-dimensional grid of processing elements (PEs) connected by programmable interconnect. Since the 1990s, extensive research has been conducted on CGRAs, and a wide range of designs have been proposed by the academic community (e.g. [4]), government (e.g. [15]) and industry (e.g. [14]). The primary ways in which CGRAs differ from each other include their logic, I/O capabilities, interconnect, and whether they support dynamic reconfigurability [10].

In this work, we consider CGRA mapping of *single-context* and *multi-context* CGRA architectures that support *predication*. *Multi-context* CGRAs allow multiple configuration bitstreams to be pre-loaded into the CGRA, which the CGRA cycles through in round-robin style (i.e. time multiplexing the CGRA resources). Each bitstream is referred to as a context, and a context completely defines the function of the CGRA's processing elements and interconnect. *Predication* is an approach to facilitate the implementation of applications with control flow on a CGRA. The predicates are true/false Boolean conditions that are used to determine whether an instruction should be executed or not, as well as to select data from one of two execution paths.

We present an architecture-agnostic CGRA mapper that supports multi-context, and predication architecture features, offering flexibility to target various architectures. The mapper extends prior work done by Ragheb et al. [13], incorporating two enhancements to accelerate the placement and routing processes within mapping. The first addition is clustering, which is used as a preprocessing step before placement and routing. Clustering, hence *CLUMAP*, ensures that interconnected operations are placed together, leading to a more compact placement. This allows simulated annealing to find solutions that utilize fewer CGRA PEs, freeing up more resources throughout the CGRA to be utilized. The clustering step eases the mapping process for intricate PE architectures, particularly those containing PEs that support predication. The second enhancement introduces a new cost function for simulated annealing placement that accounts for the operation scheduling. The cost function improves the utilization of the CGRA interconnect, leading to more efficient routing by balancing interconnect demand across contexts.

2 Background

2.1 Dataflow Graph (DFG) and Device-Model

Graph

When mapping an application to a CGRA, the graph representations of the application and the CGRA device (to which the application is mapped) are two key data structures. The mapping problem can be reduced to finding the subset of the architectural graph into which the application graph is embedded. The dataflow graph (DFG), a directed graph, is the graph representation of an application. The edges of the DFG indicate data transfer from the source (producer) operation to the sink (consumer) operation, while each node represents an operation to be executed. The input, output, load, and store

Also uses
PathFinder
for CGRA
routing step.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions.acm.org.

DAC '24, June 23–27, 2024, San Francisco, CA, USA.

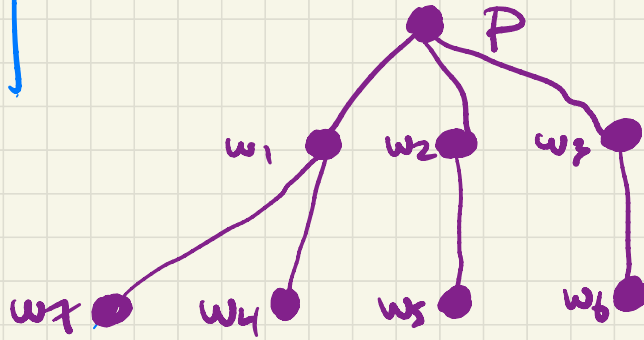
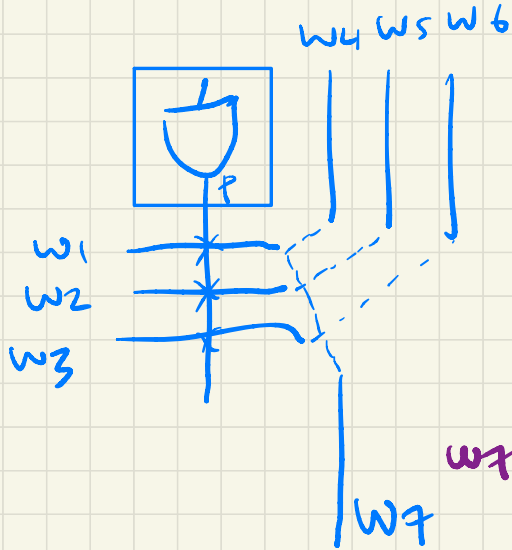
© 2024 Copyright held by the owner/auth(s). Publication rights licensed to ACM.

ACM ISBN 978-8-4061-1243-6.

<https://doi.org/10.1145/3649329.3658269>

Path Finder represents FPGA routing architecture as a graph $G(V, E)$ called Routing Resource RRG graph

Conductors (pins, wires) prog. Switches



Route connection $\longleftrightarrow$ ^{equivalent to} Find paths thru graph $\rightarrow$ graph search

With each node n in RKG Pathfinder associates $\rightarrow$ conductor

① C_n : congestion cost of n

② d_n : delay cost of n

We discuss in two parts:

① How it deals with congestion

② How to make it timing-driven
(make it deal with delay)

Negotiated Congestion Approach

What is a main weakness of last week's maze routing?

- depends on the order connections are routed
- earlier connections "block" later connections

Nair's iterative approach (1987)

Ravi Nair IBM research

IEEE TCAD

call this iter #1

1) Route all nets

- net 1 // sees "empty" chip

- net 2 // sees net 1

- net 3 // sees nets 1, 2

...

- net n // sees nets 1-n-1

2) Route all nets

- ripup and reroute net 1 // sees nets 2-n

- " " " " 2 // sees net 1, 3-n

...

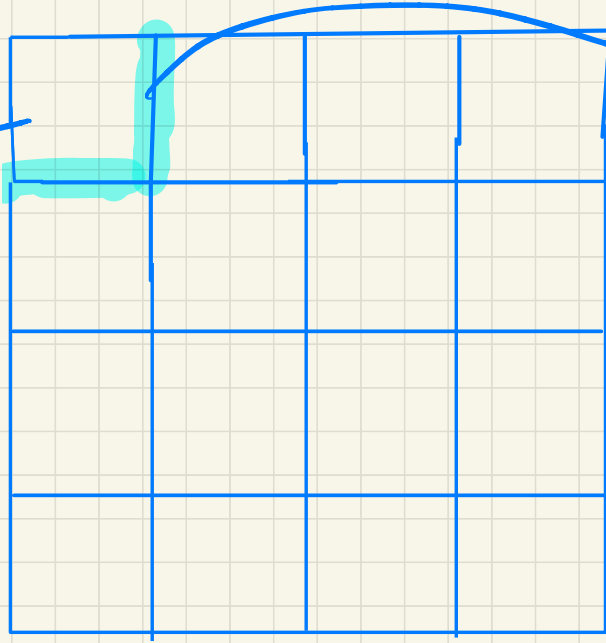
iter #2

Continue these iterations of
ripup & reroute all nets

— Makes it much less order dependent

Nair's approach allowed "illegal" states

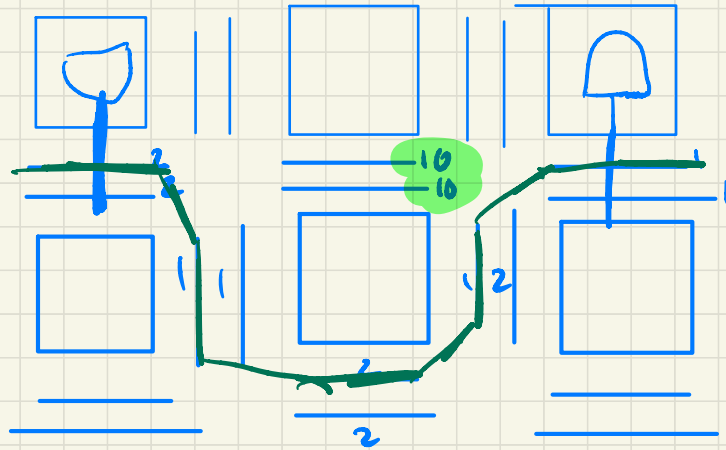
ASIC
chip
divided
into bins



We know
how many
wires can
cross bin edge
(based on
wire pitch
rules)

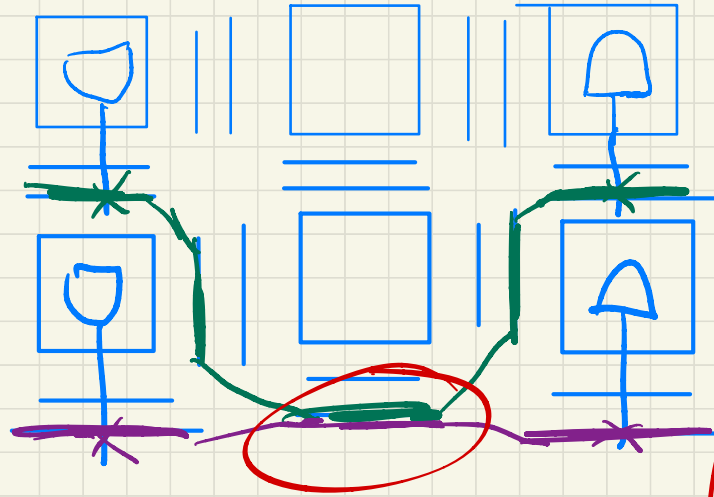
— pendlyze over congested
bin edges during routing.

PathFinder does cost-driven
maze routing $\Rightarrow$ expansion is
a priority queue



high
cost nodes!

PathFinder allows shorts in intermediate states $\Rightarrow$ removes shorts thru costing, rip-up and re-route.



Short!

(electrically wrong!)

PathFinder C_n definition ● change as

$$C_n = (b_n + h_n) \times p_n$$

rating proceeds

b_n : base cost of using n (never changes)
e.g. could be based on delay, or length,
or,

p_n : present congestion on n .
(tied to shorts on n)

h_n : history of congestion on n .

$P_n = 1$ on first Nair iteration

$P_n = (1 + \# \text{ of shorts on } n) \times P_{\text{fac}}$
on subsequent Nair iterations

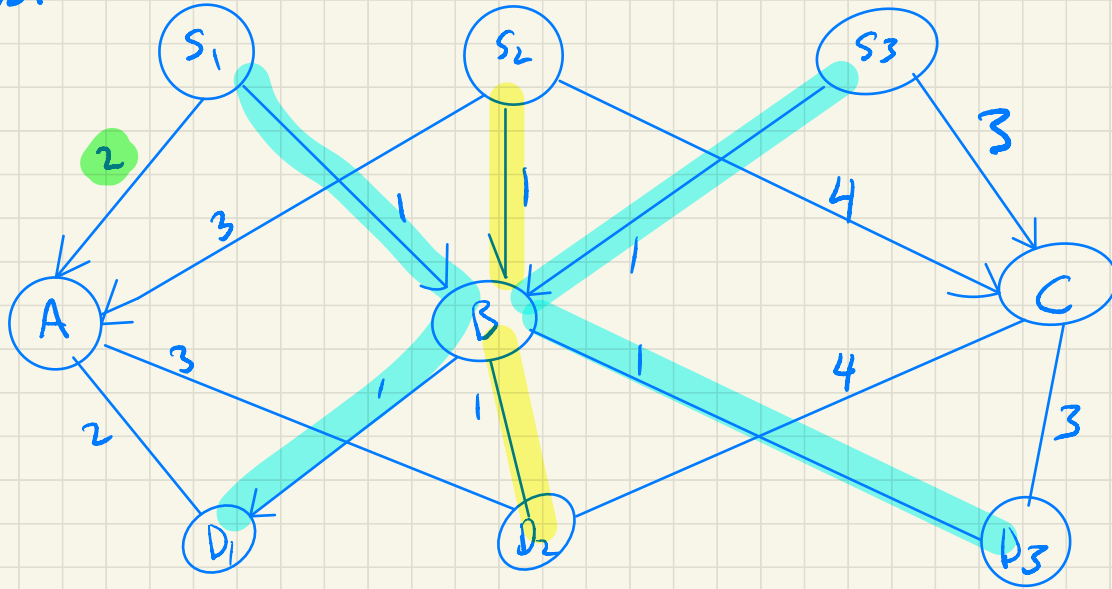
P_{fac} is a scalar weight that
is bumped up (e.g. by 2x)
each Nair iteration

why? to make it more costly to
create shorts.

Example of Pn utility

S_1, S_2, S_3 : sources
 D_1, D_2, D_3 : destinations

● Path cost



Want to connect

$S_1 \rightarrow D_1$	} 3 connections to route
$S_2 \rightarrow D_2$	
$S_3 \rightarrow D_3$	

Behaviour

- Initially $\left. \begin{array}{l} S_1 \rightarrow D_1 \\ S_2 \rightarrow D_2 \\ S_3 \rightarrow D_3 \end{array} \right\}$ all shorted on $\textcircled{B}$
- gradually increase Pfac
- eventually $S_1 \rightarrow D_2$ is routed thru $\textcircled{A}$ (path thru $\textcircled{A}$ becomes a cheaper alternative)
- eventually $S_3 \rightarrow D_3$ is routed thru $\textcircled{C}$

Then, all shorts are eliminated
⇒ PathFinder terminates
⇒ success!

(UBC)
Steve Wotton had recent FCCM
paper that used ML to
predict if routing is possible
partway thru PathFinder

P_n deals with the present congestion
(Shorts) on node n

h_n : history of congestion on n

h_n^i : h_n in iteration i

h_n^1 : 1

$h_n^i = h_n^{i-1} + (\# \text{ of shorts on } n) \times h_{\text{fac}}$

scalar
weight

h_n is a monotonically increasing penalty for nodes that had shorts in past iterations

→ Motivation is to prevent oscillations in the short removal process